Intelligent Placement Assistant:

A Retrieval-Augmented Generation (RAG) System for Campus Recruitment Support

Prototype Implementation Report

Oshish Madhan

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology
Patiala, Punjab, India
Email: omadhan_be24@thapar.edu

Course: UCS503P – Capstone Project / Prototype Stage

Submitted to:

Dr. Paramveer Sandhu

October 30, 2026

Contents

Abstract **1**

1 Higher-order Goal and Motivation **1**

2 Prototype Scope and Status **2**

2.1 Implemented in the current prototype 2

2.2 Not claimed as complete 2

3 Problem Statement **2**

4 Dataset and Quantitative Profile **3**

4.1 Dataset summary 3

4.2 Question categories 3

4.3 Recruitment batches and high-volume companies 3

5 Proposed and Implemented Solution **4**

5.1 Overview 4

5.2 Main user-facing capabilities 4

5.3 System workflow 5

6 System Architecture **5**

6.1 Technology stack 5

7 Implementation Details **6**

7.1 Document preparation and stable identifiers 6

7.2 Embedding generation 7

7.3 Persistent ChromaDB indexing 7

7.4 Dual-mode retrieval 7

7.5 Grounded LLM generation and fallback behavior 8

7.6 Analytics engine 8

7.7 FastAPI REST backend 8

7.8 React and TypeScript frontend 9

8 Evaluation and Testing **9**

8.1 Automated test suite 9
8.2 Reported performance benchmarks 10
8.3 Recommended retrieval-quality evaluation 10

9 Software Engineering Practices 11
9.1 Reproducible development commands 11
9.2 Repository 12

10 Security, Reliability, and Limitations 12
10.1 Current safeguards 12
10.2 Limitations 12

11 Scalability and Future Scope 13
11.1 Resume-to-job matcher 13
11.2 Interactive coding sandbox 13
11.3 Deployment and operational improvements 13
11.4 Retrieval and evaluation improvements 13

12 Conclusion 13

References 14

A Prototype Summary at a Glance 14

Abstract

Campus recruitment preparation often requires students to search across fragmented interview records, spreadsheets, forum posts, and other historical placement resources. Generic language models may provide plausible but unverified answers because they do not inherently have access to institution-specific recruitment archives. This project presents a prototype Intelligent Placement Assistant that retrieves evidence from a local dataset of historical campus interview questions and uses that evidence to support question answering and placement-preparation exploration.

The current prototype contains 9,302 individual questions associated with 212 companies, 27 job roles, and 317 company–role recruitment events. The backend uses the `sentence-transformers/all-MiniLM-L6-v2` embedding model and a persistent ChromaDB vector store. A dual-mode retrieval engine supports semantic similarity search as well as comprehensive metadata and keyword scans. Grounded response generation is implemented through the Google GenAI SDK, with a local fallback when the API is unavailable or unconfigured. A FastAPI service exposes chat, question-library, filter, health, and analytics functionality, while a React 19 and TypeScript frontend provides an AI Assistant, Question Library, and Interview Insights interface.

The reported automated test suite contains 47 tests, all passing at the time of the recorded run. Reported benchmarks include 12–18 ms for top-five vector queries, 20–35 ms for a full-dataset keyword scan, 2–4 ms for cached analytics responses, and 1.1–2.2 seconds for end-to-end LLM generation. This report documents the implemented prototype, its architecture, evaluation evidence, limitations, and planned extensions. The figures are implementation-reported measurements and should be reproduced under a documented test environment before being treated as independently validated results.

Keywords: Retrieval-Augmented Generation, RAG, campus placement, semantic search, ChromaDB, sentence embeddings, Gemini, FastAPI, React.

1 Higher-order Goal and Motivation

The higher-order goal is to make institution-specific recruitment information easier to find and use. Campus placement resources can be distributed across historic question collections, interview experiences, spreadsheets, and online discussions. Students may spend substantial time finding the questions associated with a particular company, role, recruitment batch, or technical topic.

A general-purpose LLM can explain common computer-science concepts, but it cannot be assumed to know the contents of a private or locally maintained campus-recruitment archive. When asked what a particular company asked during a specific campus drive, a model without the relevant evidence may produce generic examples that sound credible but do not represent verified historical records.

The prototype addresses this gap by retrieving relevant records from a local dataset before producing a response. It also exposes the question collection through a searchable library and summarizes patterns through an analytics view. The system is intended as a preparation aid rather than a guarantee of future interview questions or recruitment outcomes.

2 Prototype Scope and Status

The initial project proposal described a broader placement platform, including document upload, resume analysis, eligibility checking, and a placement dashboard. The current implementation has a more focused scope: a historical interview-question knowledge base, grounded question answering, comprehensive question search, and recruitment-topic analytics.

2.1 Implemented in the current prototype

- Ingestion and normalization of the supplied historical question dataset.
- Atomic question documents with deterministic identifiers and flat metadata.
- Dense embedding generation using Sentence Transformers.
- Persistent ChromaDB indexing with idempotent upserts.
- Semantic retrieval and comprehensive metadata/keyword retrieval.
- Gemini-based grounded generation with a local evidence fallback.
- FastAPI endpoints for chat, questions, filters, health, and analytics.
- React and TypeScript interface with three principal views.
- Cached analytics aggregations and automated tests.

2.2 Not claimed as complete

Resume-to-job matching, an interactive coding sandbox, production authentication, Docker-based deployment, and institutional cloud deployment are future extensions. The original proposal's broader PDF-upload, resume-analysis, and company-eligibility features should not be considered implemented solely because they appeared in the initial proposal.

3 Problem Statement

Students preparing for campus recruitment may need to answer questions such as:

- What historical technical questions are recorded for a specific company?
- Which topics appear frequently in the collected interview-question data?
- What question records are associated with a role or recruitment batch?
- What interview-stage information and advice is available in recorded candidate experiences?

The central engineering challenges are fragmented information, efficient retrieval across thousands of records, the need to distinguish exact historical evidence from generic model knowledge, and the need to support both focused semantic questions and exhaustive requests. Standard top-K vector search is useful for finding relevant examples but does not, by itself, guarantee that every record matching a request will be returned. The prototype therefore combines semantic retrieval with a separate comprehensive search path.

4 Dataset and Quantitative Profile

The system uses the TietPrep historical placement-question dataset located in the project repository under `code/backend/data/raw/`. The following values are the dataset metrics supplied with the implementation dossier.

4.1 Dataset summary

Table 1: Core dataset profile

Measure	Count
Atomic interview questions	9,302
Corporate recruiters / companies	212
Distinct job roles	27
Company–role recruitment events	317
Candidate-experience logs	16

4.2 Question categories

4.3 Recruitment batches and high-volume companies

The candidate-experience file, `interview_experiences.json`, is reported to contain 16 candidate logs describing interview stages, technical rounds, and advice. The supplied dossier describes the offer outcomes in these records as verified; this report does not independently audit those records. Dataset counts and percentages should be regenerated from the source files whenever the dataset is updated.

Table 2: Question category distribution

Category	Questions	Share
Technical questions	5,108	54.9%
Additional practice / OA coding questions	2,536	27.3%
HR and behavioral questions	1,658	17.8%
Total	**9,302**	**100%**

Table 3: Questions by target cohort batch

Batch	Questions
2026	6,099
2027	2,904
2028	299
Total	9,302

Table 4: Companies with the largest recorded question volumes in the supplied dataset summary

Company	Questions
American Express	321
Deloitte	225
BlackRock	146
IBM	146
ProcDna	138

5 Proposed and Implemented Solution

5.1 Overview

The implemented solution is a local web application built around a Retrieval-Augmented Generation pipeline. The question dataset is transformed into independent records, embedded, and indexed in ChromaDB. When a user asks a question, the backend selects an appropriate retrieval mode, prepares evidence, and either asks the configured Gemini model to produce a grounded answer or returns the retrieved evidence in local fallback mode.

5.2 Main user-facing capabilities

1. **AI Assistant:** conversational questions with optional company, role, category, and other retrieval filters; answers include expandable source records.
2. **Question Library:** paginated access to the question bank with keyword search, faceted filters, clipboard copying, and an action to ask the AI about a selected question.

3. **Interview Insights:** aggregate views of frequently occurring topics, recruiter question volumes, category distribution, and candidate advice.

5.3 System workflow

1. Load and normalize the source records.
2. Convert each question into an atomic document and attach scalar metadata.
3. Generate a 384-dimensional embedding for each question.
4. Upsert documents and embeddings into persistent ChromaDB storage.
5. Receive a user query through the FastAPI backend.
6. Use semantic similarity search or comprehensive filtered scanning, depending on the query and filters.
7. Construct a context-grounded prompt for the LLM, when available.
8. Return the answer, source evidence, and relevant metadata to the frontend.

6 System Architecture

The architecture separates data preparation, embeddings, vector indexing, retrieval, language-model generation, analytics, API services, and the frontend. This separation makes the retrieval and generation behavior easier to test independently.

6.1 Technology stack

Layer	Technology / implementation detail
Programming language	Python for backend and data pipeline
Embedding model	`sentence-transformers/all-MiniLM-L6-v2`
Vector database	ChromaDB, persistent local collection
Similarity metric	Cosine distance (ChromaDB HNSW space configured as `cosine`)
LLM integration	Google Gemini API via `google-genai` SDK; configured model identifier in the implementation dossier: `gemini-3.5-flash`
Backend API	FastAPI, Uvicorn, Pydantic v2 schemas
Frontend	React 19, TypeScript, Vite, Lucide Icons
Testing	Python native `unittest` framework
Version control	Git and GitHub

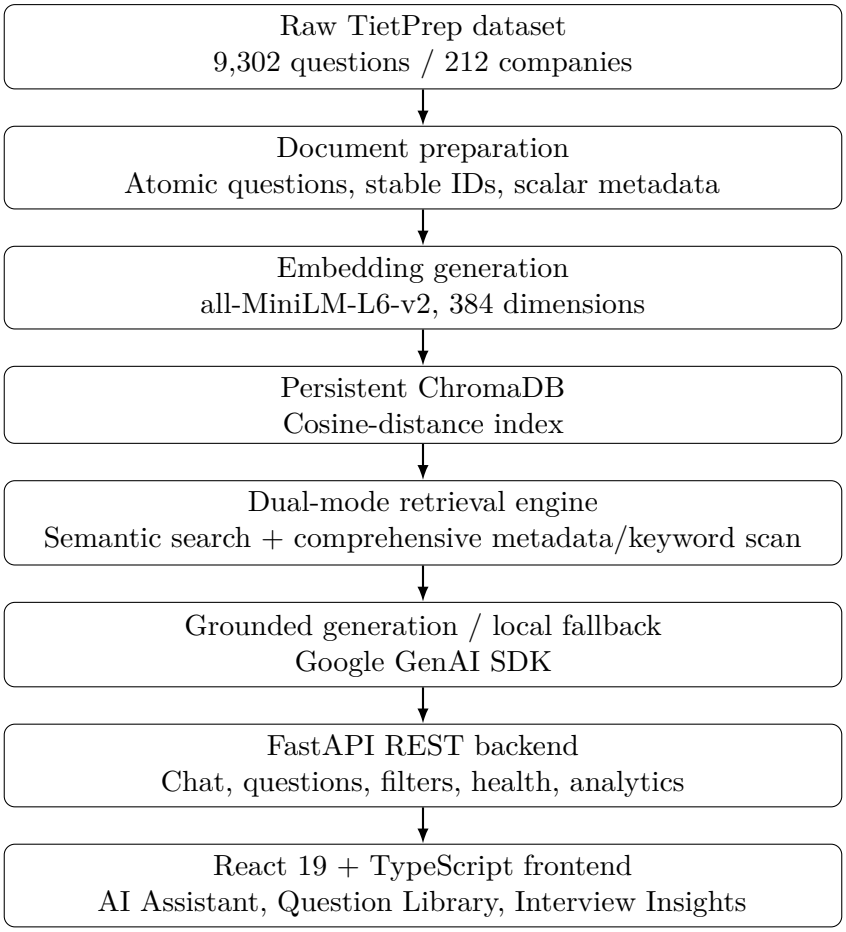


Figure 1: Implemented prototype architecture. Analytics is computed by a separate in-memory engine and exposed through the API.

7 Implementation Details

7.1 Document preparation and stable identifiers

The ingestion pipeline is implemented in `code/backend/ingestion/prepare_documents.py`. Each individual question is represented as one atomic document rather than splitting a large text blob into arbitrary token windows. This design keeps question boundaries intact and allows a retrieved result to correspond to a discrete historical question.

A deterministic identifier is formed from the source record identifier and question index, using the pattern `{record_id}_q{index}`. Stable IDs support reproducible indexing and idempotent upserts. Metadata is flattened to primitive scalar values such as strings and integers because nested objects and null metadata values are not accepted by the configured ChromaDB metadata interface.

7.2 Embedding generation

The module `code/backend/ingestion/embeddings.py` uses `sentence-transformers/all-MiniLM-L6` It produces dense vectors with 384 dimensions. The implementation dossier reports L2-normalized vectors, an in-memory singleton model cache, and batch processing. The cache avoids reloading the model weights for every query.

For a vector v, L2 normalization is expressed as

$$\hat{v} = \frac{v}{\|v\|_2}, \qquad \|\hat{v}\|_2 = 1.$$

Normalization is useful for consistent vector comparison; the actual search metric configured for the ChromaDB collection is cosine distance.

7.3 Persistent ChromaDB indexing

The indexing script is `code/backend/ingestion/index_chroma.py`. The persistent store is located at `code/backend/data/chroma_db` and is excluded from version control. The reported index run processed all 9,302 questions in 73 batches of up to 128 items and completed in 33.2 seconds. Idempotent upserts are intended to prevent duplicate records when the indexing script is rerun.

7.4 Dual-mode retrieval

The retriever, `code/backend/retrieval/retriever.py`, supports two complementary modes.

Semantic similarity mode. The user query is embedded and submitted to ChromaDB. If the returned cosine distance is d, the implementation converts it to a bounded similarity score using

$$s = \max(0.0, 1.0 - d).$$

This score is used as a display-friendly match indicator. It should not be interpreted as a calibrated probability that the result is correct.

Comprehensive search mode. For requests that need all records matching a company, topic, or question type, a top-K semantic search may omit relevant records by design. The comprehensive path scans filtered records and applies case-insensitive keyword/substring matching with pagination. The supplied implementation description characterizes this path as zero-truncation for matching records. In practice, completeness depends on the correctness of filters, matching rules, and pagination, so regression tests should continue to verify those conditions as the dataset grows.

7.5 Grounded LLM generation and fallback behavior

The generation module is `code/backend/llm/generator.py`. It uses the Google GenAI SDK with the model identifier recorded in the implementation dossier as `gemini-3.5-flash`. The prompt instructs the model to use the supplied retrieved evidence, attribute questions to company, role, and batch, and acknowledge when a company is absent from the dataset.

If the Gemini API key is missing or a network/API failure prevents generation, the application is designed to fall back to a local retrieval response rather than crashing with an HTTP 500 error. In this mode, the user receives retrieved evidence without a generated answer. Error handling also redacts API keys from error traces. Grounding instructions reduce the risk of unsupported answers but cannot mathematically guarantee that every generated statement is correct; source inspection and evaluation remain important.

7.6 Analytics engine

The analytics module, `code/backend/analytics/analytics.py`, computes aggregations used by the Interview Insights page. The supplied metrics include topic frequencies, company question-volume rankings, category distributions, and candidate advice. Aggregations are precomputed and cached in memory; reported analytics API response times are 2–4 ms.

The supplied topic counts displayed in the interface are SQL (148), Python (115), ML (94), DSA (70), and System Design (63). These figures represent the current implementation's topic-count output and should be recomputed whenever the underlying data or topic-matching rules change.

7.7 FastAPI REST backend

The API is implemented in `code/backend/main.py` and is configured for local development on port 8000. The principal endpoints are summarized below.

Endpoint	Purpose
`GET /api/health`	Health status, vector-store count, and model/API configuration status
`POST /api/chat`	Grounded RAG chat with query filters and message history
`GET /api/questions`	Paginated question bank with search and faceted filters
`GET /api/questions/{id}`	Retrieve a single question by deterministic ID
`GET /api/filters`	Return available companies, roles, batches, and categories

`GET /api/analytics/dashboard`	Return aggregated analytics payload

Cross-Origin Resource Sharing (CORS) is configured to support the local Vite frontend at `http://localhost:5173`. The backend's development command is:

```
uvicorn code.backend.main:app --host 0.0.0.0 --port 8000 --reload
```

7.8 React and TypeScript frontend

The frontend is located in `code/frontend/` and uses React 19, TypeScript, Vite, and Lucide Icons. The reported design uses a dark obsidian background (`#0b0f17`). It contains exactly three principal views:

- **AI Assistant:** conversational interface, retrieval filters, starter prompts, and expandable source pills showing verbatim questions and match percentages.
- **Question Library:** search and browse the 9,302-question bank, filter by metadata, copy questions, and pass a selected question to the assistant.
- **Interview Insights:** topic bars, company leaderboards, category distributions, and candidate advice. Selecting a chart topic can filter the Question Library.

The reported frontend production build completed in 609 ms and produced an 81.5 KB compressed JavaScript bundle. These are build measurements from the supplied project dossier, not independent measurements performed while preparing this report.

8 Evaluation and Testing

8.1 Automated test suite

The project uses Python's built-in `unittest` framework. The supplied implementation summary reports 47 tests across seven modules, with all tests passing in the consolidated run.

Test module	Coverage	Tests
`test_prepare_documents.py`	Schema, atomic chunking, ID stability	9
`test_embeddings.py`	Vector dimensions, normalization, batching	10
`test_index_chroma.py`	Collection setup, idempotent upserts	5
`test_retriever.py`	Similarity scoring, comprehensive scan, pagination	6

`test_generator.py`	Grounding prompt, fallback, key redaction	3
`test_analytics.py`	Topic counts, company aggregation, cache	3
`test_api.py`	FastAPI endpoint integration	11
Total	**Consolidated suite**	**47**

The reported pass rate is 100% (47 of 47 tests) for the recorded run. A passing unit and integration suite is useful evidence of implementation correctness for covered cases, but it does not by itself establish retrieval relevance, factual accuracy, user satisfaction, security under adversarial use, or production readiness.

8.2 Reported performance benchmarks

Table 8: Performance figures supplied for the current prototype

Operation	Reported time
Top-five vector similarity query	12–18 ms
Full-dataset keyword scan (9,302 records)	20–35 ms
Cached analytics API response	2–4 ms
End-to-end LLM generation	1.1–2.2 s
ChromaDB indexing of 9,302 questions	33.2 s
Frontend production build	609 ms

These measurements were supplied in the technical dossier. The original machine specifications, warm-up procedure, number of repetitions, percentile distribution, and exact benchmark scripts were not provided. The numbers should therefore be presented as reported prototype measurements, not as a controlled comparative benchmark. For a stronger final evaluation, record the machine and software versions, run each operation repeatedly, and report median and tail latency.

8.3 Recommended retrieval-quality evaluation

The current dossier gives implementation and latency metrics but does not provide a labeled retrieval-quality test set or independently measured answer-grounding score. The next evaluation step should create a representative set of queries with manually reviewed relevant question IDs and measure:

- **Precision@K:** proportion of the top-K retrieved records judged relevant.
- **Recall@K:** proportion of known relevant records found in the top-K results.
- **MRR or nDCG:** ranking quality when relevant records appear at different positions.

- **Groundedness:** whether each factual claim in an answer is supported by retrieved evidence.
- **Abstention behavior:** whether the system appropriately reports insufficient evidence for absent companies or unsupported questions.

These are recommended future measurements, not completed experimental results.

9 Software Engineering Practices

The implementation applies several practices intended to improve maintainability and reproducibility:

- Modular separation of ingestion, embeddings, retrieval, generation, analytics, API, and frontend code.
- Stable document identifiers and idempotent indexing.
- Environment-variable configuration for sensitive API keys, with an example environment file.
- Exclusion of local virtual environments, secrets, and vector-store binaries from version control.
- Automated unit and API integration tests.
- Pydantic schemas for API data validation.
- Error handling and API-key redaction in error traces.
- Local-development configuration for the React/Vite frontend and FastAPI backend.

The project intentionally uses the native Sentence Transformers, ChromaDB, and Google GenAI libraries instead of a higher-level orchestration framework such as LangChain. This keeps the retrieval and generation flow explicit and makes the implementation easier to inspect and test. This is an architectural trade-off rather than a claim that orchestration frameworks are inherently unsuitable.

9.1 Reproducible development commands

The commands below are those supplied for the project. Run them from the repository root, with the project's virtual environment and dependencies installed.

```
# Start the backend
uvicorn code.backend.main:app --host 0.0.0.0 --port 8000 --reload

# Start the frontend in a separate terminal
cd code/frontend
npm run dev
```

```
# Run the backend test suite from the repository root
.venv/bin/python3 -m unittest discover -s code/backend -p "test_*.py"
```

9.2 Repository

The source repository is hosted on GitHub at:

https://github.com/oshish/AI-placement-assistant-rag

The project dossier identifies the working branch as `master`. Repository state may change after the preparation of this report.

10 Security, Reliability, and Limitations

10.1 Current safeguards

The implementation uses environment variables for API credentials, redacts keys from error traces, and provides a local fallback when the Gemini API cannot be used. Retrieved source questions are exposed in the interface so users can inspect the evidence behind a response.

10.2 Limitations

- **Historical-data limitation:** the system reports what is present in its dataset; historical questions do not guarantee future interview questions.
- **Dataset coverage:** only the supplied records are available to the local knowledge base. Missing, outdated, or mislabeled records can affect answers.
- **Substring matching:** comprehensive keyword matching can miss synonyms or include false positives depending on the query and matching rules.
- **Embedding limitations:** semantic similarity may rank conceptually related but non-answering questions highly.
- **LLM limitations:** prompt constraints lower the risk of hallucination but cannot eliminate it. Generated statements should be checked against the visible source records.
- **Local prototype:** the supplied setup describes local development. Production authentication, authorization, rate limiting, deployment hardening, and multi-user load testing are not established by the current metrics.
- **Benchmark reproducibility:** hardware and repeated-run methodology were not included in the supplied measurements.

11 Scalability and Future Scope

The current modular structure provides a foundation for adding functionality. The following items are planned extensions and are not described as complete in this prototype report.

11.1 Resume-to-job matcher

Parse a student's resume, extract skills and project keywords, compare them with the question topics and role requirements available in the dataset, and identify areas that may require additional preparation. The output should be framed as preparation guidance rather than a definitive hiring prediction.

11.2 Interactive coding sandbox

Add an in-browser code editor for practicing retrieved programming questions. Potential extensions include language selection, sample test cases, execution isolation, and feedback. Any code execution service would require careful sandboxing and resource limits.

11.3 Deployment and operational improvements

Containerize the frontend and backend with Docker, define reproducible deployment configuration, and evaluate an institutional or cloud hosting environment. Before public deployment, add suitable authentication and authorization, request limits, logging and monitoring, secret management, backup/recovery procedures, and privacy controls.

11.4 Retrieval and evaluation improvements

Potential technical improvements include hybrid retrieval combining lexical and dense search, cross-encoder reranking, synonym-aware topic matching, dataset-quality checks, a manually labeled evaluation set, and more systematic groundedness testing. These changes should be compared against the existing implementation using the same evaluation queries.

12 Conclusion

This prototype implements an Intelligent Placement Assistant focused on historical campus interview questions. It transforms 9,302 question records into a persistent vector index, supports both semantic retrieval and comprehensive filtered search, and provides a grounded generation path with a local evidence fallback. A FastAPI backend connects

the retrieval, generation, and analytics modules to a React and TypeScript interface containing an AI Assistant, Question Library, and Interview Insights view.

The supplied implementation dossier reports 47 passing automated tests and low retrieval and analytics latencies, with LLM generation taking approximately 1.1–2.2 seconds. These figures demonstrate the reported state of the prototype but should be reproduced with documented test conditions and supplemented by retrieval-quality and answer-grounding evaluation. Future work will focus on stronger evaluation, deployment hardening, and extensions such as resume-to-job matching and a coding-practice sandbox. The system is intended to help students explore historical evidence and prepare more efficiently, not to predict or guarantee the content of future recruitment rounds.

References

[1] Lewis, P. et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Reimers, N. and Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of EMNLP-IJCNLP*, 2019.

[3] Sentence Transformers Documentation. https://www.sbert.net/

[4] ChromaDB Documentation. https://docs.trychroma.com/

[5] Google AI for Developers. https://ai.google.dev/

[6] FastAPI Documentation. https://fastapi.tiangolo.com/

[7] React Documentation. https://react.dev/

[8] Vite Documentation. https://vite.dev/

[9] Project repository: Intelligent Placement Assistant RAG. https://github.com/oshish/AI-placement-assistant-rag

A Prototype Summary at a Glance

Item	**Current reported state**
Dataset	9,302 questions, 212 companies, 27 roles
Vector representation	384-dimensional Sentence Transformer embeddings
Vector store	Persistent local ChromaDB
Retrieval	Semantic top-K and comprehensive filtered keyword search
Generation	Gemini API with local retrieval fallback
Backend	FastAPI on local port 8000
Frontend	React 19 + TypeScript + Vite on local port 5173
Main views	AI Assistant, Question Library, Interview Insights
Automated tests	47 reported passing tests
Future work	Resume matcher, coding sandbox, Docker/cloud deployment, improved evaluation